FinTwin

AI-Powered Banking Database Performance Simulator & Digital Twin

PROJECT REVIEW 1

Team Member	Registration Number
Arav Singh Patel	24BCE2425
Chaitanya Ghogare	24BCE2392

Database Systems Project

25 September 2026

GitHub Link: https://github.com/Arrav-SP/FinTwin

1. Project Overview

FinTwin is a performance digital twin for PostgreSQL databases. The project studies how a database behaves under different banking workloads and uses the collected performance data to build machine learning models that can predict database performance for new workload configurations.

The project is focused on database performance rather than banking business logic. A user can configure factors such as transaction rate, concurrency, duration and workload type. FinTwin can then predict expected latency, throughput, CPU utilization and lock contention, and the same configuration can be executed against PostgreSQL for validation.

2. Problem Statement

Database performance can change significantly as workload intensity and concurrency increase. Testing every possible workload configuration directly on a database can be time-consuming and may require repeated experiments.

The objective of FinTwin is to create a controlled environment where banking workloads can be generated, database behavior can be measured, and machine learning can be used to estimate performance for configurations that have not yet been executed.

3. Objectives

- Design a relational PostgreSQL database representing a simplified banking environment.
- Generate realistic synthetic banking transactions and execute them against PostgreSQL.
- Measure database performance using latency, throughput, CPU, memory and lock-related telemetry.
- Build machine learning models that predict performance from workload configuration.
- Provide what-if analysis for comparing different workload configurations without executing them.
- Identify potential database bottlenecks using rule-based optimization checks.
- Provide a simple frontend through which users can configure workloads and view results.
- Validate predicted results by comparing them with actual database experiments.

4. System Architecture

The current system is divided into several major components: a frontend, FastAPI backend, workload runner, telemetry collector, PostgreSQL database and machine learning pipeline.

Component	Purpose
Frontend	Allows users to configure workloads, request predictions and view experiment results.
FastAPI Backend	Provides API endpoints and coordinates predictions, experiments and validation.
Workload Runner	Generates and executes synthetic banking operations at the requested workload level.
Telemetry Collector	Collects PostgreSQL and host performance metrics during experiments.
PostgreSQL 16	Acts as the database under test and stores banking and telemetry data.
ML Pipeline	Trains models and predicts latency, throughput, CPU and lock behavior.

5. Database Design

The database currently contains six main business tables: branches, customers, accounts, merchants, transactions and loans. Additional telemetry tables are used to store experiment and performance information.

- Primary and foreign keys are used to maintain relationships between entities.
- Unique constraints are applied to fields such as branch codes, IFSC codes, email addresses and phone numbers.

- CHECK constraints enforce valid values such as non-negative account balances and positive transaction amounts.
- Transaction-specific constraints ensure that required account fields are present for different transaction types.
- Indexes have been created for foreign keys, timestamps and telemetry queries.

6. Transaction Processing and Concurrency

Banking operations such as transfers are implemented using database transactions. Transfers use row-level locking with SELECT ... FOR UPDATE. To prevent circular waiting between concurrent transfers, account IDs are locked in a consistent ascending order.

The transfer operation is wrapped in a database transaction so that failures result in rollback. This provides atomicity while the database constraints provide an additional layer of consistency.

7. Workload Simulation

The workload generator supports multiple banking scenarios with different mixes of operations. Current scenarios include normal-day activity, salary-day activity, festival spikes and high-concurrency transfer workloads.

The runner separates workload pacing from operation latency measurement. Target TPS represents the requested submission rate, while actual TPS is calculated from completed operations. This allows the system to observe the point where database capacity becomes a limiting factor.

Scenario	Example Workload
NORMAL_DAY	Balance checks and transfers representing normal activity.
SALARY_DAY	Higher proportion of deposits and balance checks.
FESTIVAL_SPIKE	Higher payment activity representing a traffic spike.
HIGH_CONCURRENCY_TRANSFER	Transfer-heavy workload intended to exercise locking and concurrency.

8. Telemetry and Performance Monitoring

A background telemetry collector samples PostgreSQL approximately once per second during experiments. The current implementation collects database activity, transaction statistics, lock information, database size and host-level CPU and memory utilization.

- Active, idle, waiting and blocked database connections.
- Transaction commits, rollbacks and deadlocks.
- Buffer cache hits and reads.
- Total and waiting locks.
- Database size.
- Host CPU and memory utilization.

9. Machine Learning Component

The ML pipeline uses workload configuration as input features. These include scenario, concurrency, target TPS, duration, burstiness and operation-mix ratios. Runtime measurements such as latency and actual TPS are used as target variables rather than input features.

Three regression approaches are currently evaluated: Linear Regression, Random Forest and Gradient Boosting. Model selection is performed separately for each target metric using Mean Absolute Error, with grouped splitting used to reduce contamination between similar configurations.

The current dataset contains only a small number of experiments, so the ML results are treated as an early proof of concept rather than production-grade predictive performance.

10. What-If Analysis

FinTwin includes a what-if engine that compares a baseline workload with a modified workload using model predictions. It calculates percentage changes in predicted metrics and categorizes the magnitude of the predicted impact.

Sensitivity analysis can also vary one workload parameter, such as concurrency, and estimate how the predicted performance changes as that parameter is increased. These results are treated as model-derived relationships rather than proof of causal effects.

11. Optimization Engine

A rule-based optimization component examines measured experiment data for potential bottlenecks. Current checks include CPU pressure, memory pressure, connection saturation, lock contention, tail-latency spikes and poor cache behavior.

Recommendations are advisory only. The system does not automatically change database configuration or modify the database based on these recommendations.

12. Frontend

A lightweight frontend has been implemented using HTML, CSS and vanilla JavaScript. It provides workload configuration, prediction and experiment-result views without external frontend frameworks or build tools.

- Dark and light themes.
- Loading states using skeleton animations.
- Explicit empty and error states.
- Charts for visualizing performance information.
- Handling of missing values without presenting fabricated zeros.

13. Testing and Current Results

Testing currently covers core database operations, concurrency behavior, workload pacing, telemetry persistence, ML feature extraction, what-if calculations, optimization thresholds, validation calculations and API contracts.

These initial validation results demonstrate that the complete prediction-to-experiment workflow is operational. The relatively high prediction errors are expected at this stage because the model has been trained on only a small number of experiments.

Metric	Predicted	Actual	Error
Average Latency	9.46 ms	18.83 ms	49.75%
P95 Latency	13.30 ms	39.73 ms	66.52%
Throughput	12.15 TPS	10.08 TPS	20.57%
Host CPU	13.13%	26.55%	50.54%
Lock Waits	0	0	0%

14. Current Limitations

- The current ML dataset is small, limiting the reliability of predictive models.
- Account selection is currently uniform, whereas real workloads may contain hot accounts and skewed access patterns.
- CPU and memory telemetry is collected at the host level rather than being isolated to PostgreSQL.

- Query-plan analysis has not yet been integrated.
- The burstiness configuration is stored but is not yet connected to the workload pacing algorithm.
- The validation endpoint currently performs experiments synchronously.

15. Planned Work

- Increase the experiment dataset substantially across different scenarios, concurrency levels and TPS values.
- Introduce more realistic, non-uniform account selection.
- Implement the burstiness parameter in workload generation.
- Improve container-aware resource monitoring.
- Integrate PostgreSQL query-plan analysis using EXPLAIN ANALYZE.
- Move long-running validation experiments to an asynchronous/background execution model.
- Evaluate additional ML models and investigate prediction confidence intervals.
- Improve the frontend with additional visualization and experiment comparison capabilities.

16. Conclusion

At the first project review stage, the major components of FinTwin have been implemented and integrated into a working end-to-end prototype. The project currently supports database schema management, transactional banking operations, workload generation, telemetry collection, machine learning based prediction, what-if analysis, optimization recommendations, frontend visualization and validation against real PostgreSQL experiments.

The primary focus for the next stage is to expand the experimental dataset and improve workload realism so that the predictive component can be evaluated more reliably. The existing system provides the foundation required for these improvements.